High-Level Design (HLD) Document

Essential Digital Services Module - AOFIDPS

Table of Contents

1. Executive Summary

Problem Statement

Economic and civic participation increasingly demands digital access. However, in low-connectivity regions, synchronous web-based services—such as applying for micro-loans, submitting agricultural yields, or accessing government subsidies—fail. A dropped 2G connection mid-submission results in timed-out forms, lost data, or worse, duplicate financial charges due to desperate user retries.

Proposed Solution

The Essential Digital Services Module of the AOFIDPS platform decentralizes transactional logic. It shifts the paradigm from synchronous "Web Requests" to asynchronous "Cryptographic Intents". Users interact with dynamically cached offline forms, validate data locally, and cryptographically sign a transaction intent. The module stages these transactions securely and guarantees exact-once execution (idempotency) upon network reconnection, bridging the gap between offline users and online financial/governmental institutions.

2. System Overview

2.1 Project Background

Traditional web forms and financial apps are inherently synchronous. They assume an unbroken pipe between the client and the server database. When applied to rural environments, this architecture causes severe data fragmentation and financial anxiety. Users need a system that acts like an "offline outbox," securing their intents and executing them flawlessly when the environment permits.

2.2 Key Objectives

- Enable 100% offline form filling and complex validation.
- Eliminate duplicate transactions (double-spending) resulting from network retries.
- Resolve data conflicts automatically when multiple offline users edit the same remote entity.
- Provide an immutable, auditable trail of queued actions and sync statuses.

2.3 Scope and Boundaries

In Scope:

- Offline UI rendering via JSON schemas.
- Local transaction tokenization and biometric signing.
- Idempotent commit protocols on the backend.
- CRDT (Conflict-free Replicated Data Type) data merging.

Out of Scope:

- Real-time stock trading or high-frequency banking (which strictly requires synchronous latency limits).
- Processing raw credit card numbers locally (relies on tokenized payment gateway integration instead).

3. Architecture Principles

3.1 Design Philosophy

- **Event Sourcing:** The system does not store the "current state" of a form; it stores the sequence of actions (events) the user took. The final state is derived by replaying these events.
- **CQRS (Command Query Responsibility Segregation):** The UI reads from a fast, local, read-only materialized view (Query). User actions (e.g., "Submit Application") are written to an isolated, secure append-only queue (Command).
- **Guaranteed Exact-Once Execution:** No matter how many times a device attempts to push a queued packet over a flaky network, the backend processes the business logic only once.

3.2 Architectural Patterns

- **Asynchronous Queueing:** Staging commands locally.
- **Idempotency Keys:** Unique hashes attached to every financial or civic write command.
- **Vector Clocks / CRDTs:** For deterministic, decentralized conflict resolution without server-side manual intervention.

4. Technology Stack

Layer	Technology	Justification
Mobile UI	Flutter & JSON Forms	Flutter renders dynamic UI components on the fly based on JSON schemas downloaded previously.
Local Data Engine	Isar DB (Edge NoSQL)	Provides ACID guarantees for the local transactional queue.
Cloud Routing	Node.js / Express	Highly concurrent async I/O routing for incoming bursts of delayed transactions.
Idempotency Lock	Redis	In-memory key-value store used to instantly check if a Transaction_UUID has already been processed.
Persistent Ledger	PostgreSQL	Strict, relational ACID-compliant database for financial and civic records.

5. System Architecture

5.1 High-Level Architecture Diagram

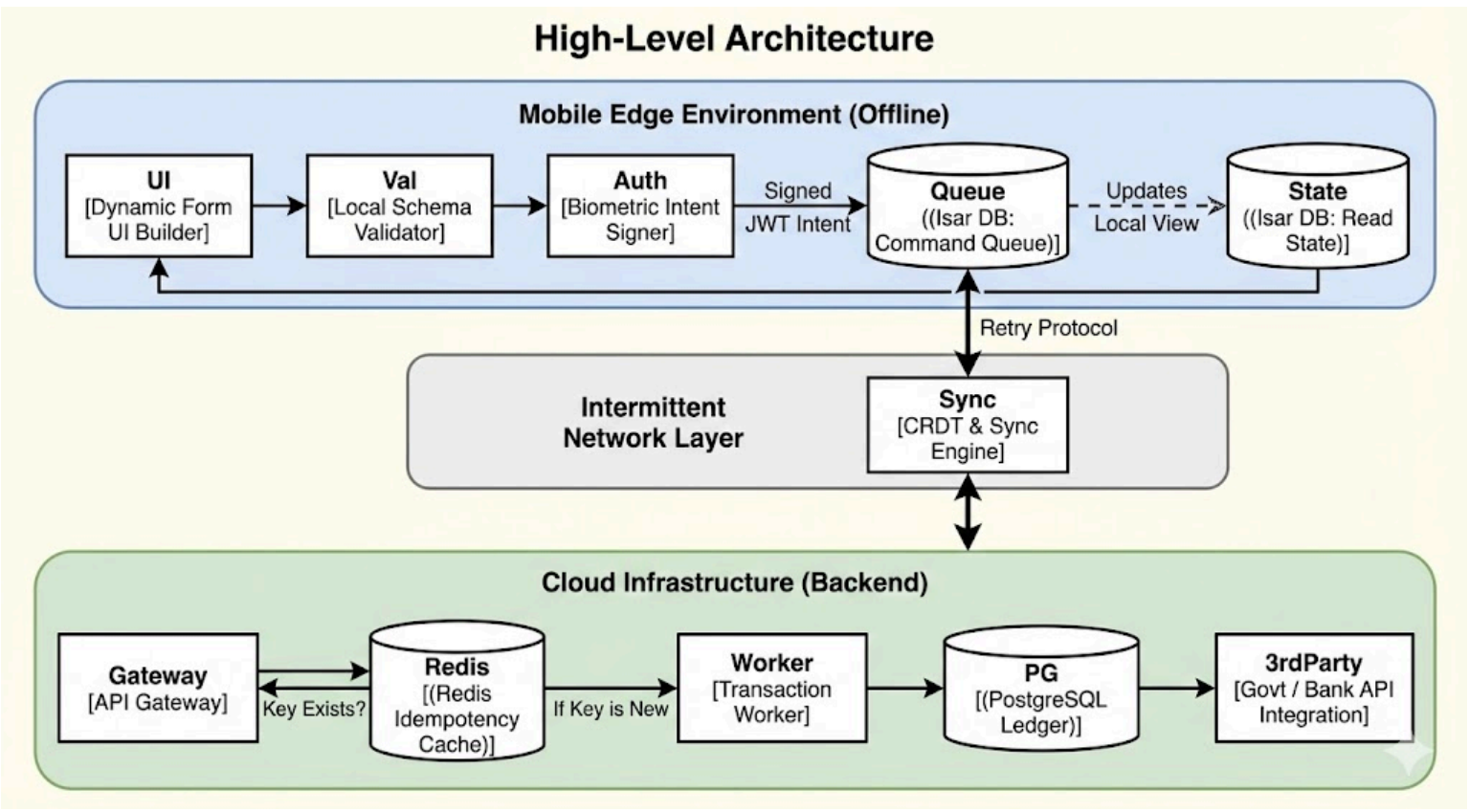
```
graph TD
  subgraph Mobile Edge Environment (Offline)
    UI[Dynamic Form UI Builder]
    Val[Local Schema Validator]
    Auth[Biometric Intent Signer]
    Queue[(Isar DB: Command Queue)]
    State[(Isar DB: Read State)]

    UI --> Val
    Val --> Auth
    Auth -->|Signed JWT Intent| Queue
    Queue -.->|Updates Local View| State
    State --> UI
  end

  subgraph Intermittent Network Layer
    Sync[CRDT & Sync Engine]
  end

  subgraph Cloud Infrastructure (Backend)
    Gateway[API Gateway]
    Redis[(Redis Idempotency Cache)]
    Worker[Transaction Worker]
    PG[(PostgreSQL Ledger)]
    3rdParty[Govt / Bank API Integration]

    Queue <==>|Retry Protocol| Sync
    Sync ==> Gateway
    Gateway --> Redis
    Redis -- Key Exists? --> Gateway
    Gateway -->|If Key is New| Worker
    Worker --> PG
    Worker --> 3rdParty
  end
```



5.2 Component Overview

- **Dynamic Form UI Builder:** Reads a cached .json file defining form fields, dropdowns, and dependencies, rendering native Flutter widgets.
- **Local Schema Validator:** Uses RegEx and logical rules to ensure data is correct before allowing the user to click "Submit" offline.
- **Biometric Intent Signer:** Wraps the validated form data into a JWT payload and signs it using the device's secure enclave (FaceID/Fingerprint).
- **Redis Idempotency Cache:** A high-speed cache storing recently processed Intent_UUIDs to intercept and block duplicate network submissions.
- **Transaction Worker:** Validates the cryptographically signed intent and executes the API call to external banks or government endpoints.

6. Data Architecture

6.1 Database Strategy (Optimized)

- **Mobile (Isar DB Partitions):**
  - Schema_Registry: Version-controlled JSON definitions of government/financial forms.
  - Intent_Queue: The offline outbox. Contains payload, UUID, Timestamp, and Sync_Status (Pending, In-Flight, Success, Failed).
  - Materialized_Views: Local representation of the user's account balance or application status, computed from synced events.
- **Cloud (PostgreSQL & Redis):**
  - Intent_Ledger: Immutable log of every transaction received.
  - Idempotency_Locks (Redis): Key: UUID, Value: Status (Processing/Done), TTL: 48 hours.

6.2 Data Flow Design (Situational)

1. **Offline Initiation:** A farmer fills out a "Crop Yield Subsidy" form in a field with no signal.
2. **Local Commit:** The app validates the data, generates a UUIDv4, signs the payload with the farmer's biometric private key, and writes it to the Intent_Queue. UI updates to "Queued for Submission."
3. **Flaky Connection (The Double-Send):** The farmer drives into town. The phone hits an unstable 3G network. The Sync Engine fires the packet. The server receives it, but the network drops before the server can send an HTTP 200 OK back to the phone.
4. **The Retry:** The phone regains connection 5 minutes later. Assuming the first packet failed, the phone automatically re-sends the *exact same packet*.
5. **Idempotent Resolution:** The backend checks the UUID against Redis. It sees the transaction was already successfully processed 5 minutes ago. It ignores the payload but resends the HTTP 200 OK receipt. The phone marks the queue item as "Success".

7. Security & Integrity Architecture

- **Non-Repudiation:** Because the transaction payload is signed by a private key residing in the device's hardware hardware-backed keystore, users cannot later claim "I didn't submit that transfer."
- **Schema Versioning:** If a government agency updates a form (e.g., adding a new tax field), the Schema_Registry versions from v1.0 to v1.1. If an offline user submits a v1.0 form, the backend includes translation logic to gracefully handle or deprecate the outdated submission without crashing.
- **Payload Tamper Resistance:** The JWT signature covers the entire JSON payload and the Timestamp. If a malicious node intercepts the packet and changes Amount: $50 to Amount: $500, the signature verification will instantly fail on the backend.

8. Deployment Architecture

- **Message Broker Isolation:** Incoming delayed transactions from millions of devices returning online simultaneously can crash a monolithic backend. The API Gateway routes incoming intents directly into a highly scalable Message Broker (e.g., Apache Kafka or RabbitMQ).
- **Asynchronous Workers:** Cloud Run worker nodes pull from the Kafka queue at a controlled pace, interacting with legacy, slow third-party Bank/Gov APIs without dropping mobile client connections.
- **Webhook Architecture:** Once a 3rd-party API finalizes a transaction, the backend pushes a silent notification (via FCM/APNs) to the mobile device to update the local read state.

9. Risk Management

Risk	Impact	Deep Mitigation Strategy
The "Double Spend"	User charged twice for one payment.	Strict API idempotency. The backend Redis cluster checks a unique, client-generated UUID for every mutation request. Duplicates yield a success receipt but bypass execution.
Split-Brain Conflicts	Two offline cooperative managers edit the same inventory ledger concurrently.	Implement CRDTs (Conflict-free Replicated Data Types) using Vector Clocks. Edits are merged deterministically (e.g., "Additions win over Deletions", or mathematically merging integer counters).
Lost or Stolen Device	Malicious actor submits unauthorized queued transactions.	Device-level biometric gating required before writing to the Intent_Queue. Remote wipe capabilities triggered via EORS mesh if the device is blacklisted.
Stale Read States	User thinks they have $100 locally, but only have $10 on the server.	Local UI explicitly labels offline balances as "Pending Sync / Estimated." Offline transactions are soft-locked locally until cloud reconciliation occurs.

10. Deep Problem Analysis with Deepest Solutions

Problem 1: Form Validation in a Vacuum

- **Deep Analysis:** Web forms usually validate data (e.g., "Does this ID number exist?") by pinging a server. Offline, this is impossible. If a user submits a 50-field form offline and finds out 3 days later that one field was formatted wrong, the user experience is catastrophic.
- **Deep Solution:** JSON-Schema driven local rules and Edge-cached lookup tables.
- **Implementation Approach:** Forms are not hardcoded in the app; they are rendered from downloaded JSON Schemas. These schemas contain deeply complex RegEx validation rules and conditional logic (e.g., "If Q1=Yes, Q2 is required"). For fields requiring database lookups (like postal codes), the system pre-fetches compressed, heavily indexed SQLite dictionary tables for local querying.

Problem 2: Network-Induced Duplicate Transactions

- **Deep Analysis:** In a 2G network environment, HTTP timeouts are rampant. A client sends a POST request, the server executes it, but the client times out waiting for the response. The client retries. The server executes it again.
- **Deep Solution:** Client-side UUID generation and server-side Idempotency Locks.
- **Implementation Approach:** The moment the user clicks "Submit", the Flutter app generates a UUIDv4 (e.g., 123e4567-e89b-12d3-a456-426614174000). This becomes the primary key of the transaction intent. The backend uses a Redis SETNX (Set if Not eXists) command. If the UUID is new, it processes the financial logic. If it exists, the transaction is safely skipped, and the success receipt is returned to clear the mobile queue.

Problem 3: Multi-Actor Concurrency Conflicts

- **Deep Analysis:** Imagine an agricultural cooperative where Manager A and Manager B are both offline. Manager A logs a harvest of 50 apples. Manager B logs a harvest of 30 apples. When both sync, a standard database would just overwrite one with the other (Last Write Wins), causing data loss.
- **Deep Solution:** State-based CRDTs (Conflict-free Replicated Data Types).
- **Implementation Approach:** The system does not sync the *state* ("There are 50 apples"). It syncs the *operation* ("Increment Apple Counter by 50"). Using CRDT logic, the operations are mathematically commutative. Regardless of the order in which Manager A and Manager B's packets arrive at the server, the final derived state will accurately be exactly +80 apples, perfectly preserving both offline intents without manual conflict resolution.

11. Glossary

- **Idempotency:** A mathematical/computer science property where applying an operation multiple times has the same effect as applying it exactly once.
- **Intent Token:** A cryptographically signed JSON payload representing a user's action, decoupled from immediate execution.
- **CRDT:** Conflict-free Replicated Data Type. A data structure that can be replicated across multiple offline nodes and mathematically guarantees conflict-free merging.
- **CQRS:** Command Query Responsibility Segregation. Separating the architecture into "Read" pipelines and "Write" pipelines.
- **Event Sourcing:** Modeling state not as a single snapshot, but as a sequential log of all events that have ever occurred.